

Matrioshka Brain — Stellar Compute Compiler + Real-Time Visualizer

Full-stack, portfolio-grade project that combines a **distributed scheduler / compiler** for a Matrioshka Brain with a **high-fidelity WebGL visualizer**. Includes architecture, roadmap, repo layout, starter code, and milestone checklist.

1. Project Overview

Goal: Build a simulated interstellar cloud compute platform powered by a Matrioshka Brain and an interactive real-time visualizer that demonstrates energy flow, job scheduling, heat dynamics, and resilience. The system will be credible (uses physics approximations), technically deep (Rust/Go + WebGPU/Three.js + shaders), and polished for demos and papers.

Core capabilities: - Physical model: star luminosity, shell absorption, thermal radiation, and energy transfer between shells. - Compute model: convert harvested energy into FLOPS and memory capacity (using thermodynamic approximations and user-definable efficiency parameters). - Scheduler: energy-aware distributed scheduler/orchestrator that allocates workloads to shells and dynamically rebalances under thermal/energy constraints. - Visualizer: real-time WebGL (Three.js or WebGPU) frontend showing nested shells, particles/flow for energy, job packets, and monitoring overlays. - Observability: Prometheus-style metrics, Grafana dashboards, and live telemetry from the simulation.

2. High-level architecture

```
flowchart LR
    subgraph Simulation_Backend [Simulation Backend]
        A[Physics Engine (Python)] --> B[Scheduler (Rust/Go)]
        B --> C[Orchestrator API (gRPC/REST)]
        A --> D[Telemetry Exporter]
    end

    subgraph Visualization
        E[Realtime Ingest (WebSocket)] --> F[Visualizer (WebGL / WebGPU)]
        C --> E
        D --> E
    end

    subgraph Observability
        G[Metrics Collector] --> H[Grafana]
        D --> G
        B --> G
    end
```

```
end

User --> C
User --> F
```

Notes: - Physics engine approximates stellar output & per-shell thermodynamics; written in Python for numerical flexibility. - Scheduler written in Rust (or Go) for performance and system-level control; implements allocation, throttling, failure simulation. - Orchestrator exposes API endpoints to submit workloads, query state, and control simulation parameters. - Visualizer connects via WebSocket for live telemetry and renders using advanced shaders for energy flow.

3. Roadmap & Milestones

Phase 0 — Foundations (1–2 weeks) - Project repo + CI - Small Python physics prototype (star + N shells energy model) - Basic static Three.js scene (nested transparent spheres)

Phase 1 — Core Simulation (3–4 weeks) - Complete physics engine (radiative transfer approximation, per-shell energy balance) - Convert energy to compute capacity (FLOPS estimate using Landauer-style approximations) - Unit tests and validation notebooks

Phase 2 — Scheduler & Orchestrator (3–5 weeks) - Scheduler design & implementation in Rust/Go - Workload models (training job, simulation job, storage job) - gRPC / REST API and job lifecycle - Fault injection support (node overheating, micro-meteorite damage)

Phase 3 — Real-Time Visualizer (3–5 weeks) - WebSocket telemetry pipeline - Advanced visuals: particle systems, shader-based flow, heat bloom - Interactive controls: pause, speed, change star type, inject jobs

Phase 4 — Observability & Polishing (2–3 weeks) - Prometheus metrics + Grafana dashboards - Demo video, README, research-style whitepaper - Deployment (host backend on cloud / demo on static front-end host)

Total estimated time: 10–14 weeks for a polished deliverable (solo dev). Add team members to shorten.

4. Starter Repo Structure

```
matrioshka-project/
├─ backend/
│   ├─ physics_engine/      # Python package
│   │   └─ pyproject.toml
│   │   └─ physics/
│   │       └─ star.py      # star model (luminosity, temp)
│   │       └─ shells.py    # shell energy balance
```

```

| | | └─ compute.py      # convert energy -> FLOPS/memory
| | └─ notebooks/       # exploratory notebooks
| └─ scheduler/         # Rust or Go project
|   └─ Cargo.toml / go.mod
|     └─ src/
|       └─ main.rs / main.go
|         └─ scheduler.rs  # scheduler core
|           └─ orchestrator.rs # API + job control
└─ telemetry/           # metrics exporter
    └─ exporter.rs

└─ frontend/
    └─ viz/              # Three.js or WebGPU app (React or vanilla)
        └─ package.json
        └─ src/
            └─ index.html
            └─ main.js
            └─ scene.js
            └─ shaders/
                └─ socketClient.js
        └─ public/
    └─ docs/              # demo content, screenshots

└─ infra/
    └─ docker/           # docker-compose for local testing
    └─ k8s/              # optional

└─ docs/
    └─ architecture.md
    └─ whitepaper.pdf
    └─ demo_script.md

└─ README.md

```

5. First Code Modules (starter templates)

I include two minimal starter modules: `physics_engine/star.py` (Python) and `scheduler/src/main.rs` (Rust). These are intentionally small but functional — they let you run a tiny sim and a scheduler loop locally.

backend/physics_engine/physics/star.py

```
# star.py – simple star model
import math

class Star:
    def __init__(self, luminosity_watts=3.828e26, radius_m=6.9634e8,
name='Sun'):
        self.name = name
        self.luminosity = float(luminosity_watts)
        self.radius = float(radius_m)

    def flux_at_radius(self, r_m):
        """Return watts per square meter at distance r (inverse square law)."""
        if r_m <= 0:
            raise ValueError('r must be positive')
        return self.luminosity / (4 * math.pi * r_m**2)

    def __repr__(self):
        return f"Star(name={self.name}, L={self.luminosity:.3e} W)"

# quick demo
if __name__ == '__main__':
    s = Star()
    print(s)
    print('Flux at 1 AU:', s.flux_at_radius(1.495978707e11))
```

backend/scheduler/src/main.rs (Rust starter)

```
// main.rs – tiny scheduler loop skeleton (requires Rust toolchain)
use std::thread;
use std::time::Duration;

fn main() {
    println!("Stellar Compute Scheduler – starter");
    // This is a simplified event loop. In production replace with async runtime
    (tokio)
    loop {
        // 1. sample telemetry (placeholder)
        println!("Polling energy state...");

        // 2. run scheduling pass (placeholder)
        println!("Running scheduling pass... allocating jobs to shells");
    }
}
```

```

    // 3. emit metrics (placeholder)
    println!("Emitting metrics...");

    thread::sleep(Duration::from_secs(2));
}
}

```

6. Important Design Decisions & Algorithms

Physics approximations

- Use inverse-square law for flux + blackbody radiative equilibrium for shell temperatures.
- Per-shell energy capture modeled as: $\text{captured} = \text{incident_flux} * \text{area} * \text{capture_efficiency}$.
- Radiative coupling between shells: treat shells as concentric blackbodies exchanging energy via radiation only.

Compute conversion

- Base estimate using conservative per-op energy: $E_{\text{op}} = 1\text{e-}21 \text{ J/op}$ (adjustable). Compute $\text{FLOPS} = \text{captured_power} / E_{\text{op}}$.
- Add memory footprint: estimate bytes/op and convert to storage requirements.

Scheduling & optimization

- Scheduler objectives: maximize throughput while keeping shell temps < threshold.
- Model as constrained optimization (mixed integer linear program for static allocation; greedy or LP-based heuristics for online allocation).
- Consider fairness or priority classes for jobs.

Fault model

- Overheat → shell throttles or shuts down.
- Random damage events that remove shells permanently; scheduler must re-allocate workload.

7. UX / Visualizer Features (what to demo)

- Live nested shells with per-shell color-coded heat map.
- Particle flows representing energy packets moving between shells.
- Job packets as small spheres traveling to assigned shells and changing color while executing.
- Heat bloom post-processing (bloom/blur) for photorealism.
- Side panel with metrics, charts (Plotly or D3), and controls (inject job, vary star, pause/resume).

8. Milestone Checklist (concrete tasks)

M0: Repo & MVP - [] Init mono-repo (git + CI) - [] Python physics: star + shells model + unit tests - [] Minimal Rust scheduler skeleton - [] Simple Three.js scene with nested spheres

M1: Physics complete - [] Energy capture per shell - [] Compute conversion function - [] Notebook with validation cases

M2: Scheduler & API - [] Scheduler core (allocation pass) - [] API to submit jobs + query status - [] Telemetry exporter

M3: Visualizer & Telemetry - [] WebSocket telemetry pipeline - [] Real-time rendering of flows & jobs - [] Interactive controls

M4: Hardening & UX - [] Prometheus metrics + Grafana dashboards - [] CI building + artifact for frontend - [] Demo video & write-up

9. Demo scripts & talking points

Elevator pitch (30s):

"I built an interstellar, energy-aware cloud compute prototype: a scheduler that treats nested Dyson shells as compute nodes, dynamically allocating workloads under thermodynamic constraints and visualizing the entire system in real-time using WebGL. It combines physics, distributed systems, and real-time graphics to explore what computation looks like at stellar scale."

Deep-dive (3–5 min): - Show the physics engine notebook, explain energy→compute conversion, discuss assumptions (Landauer, efficiencies). - Show scheduler code and explain allocation heuristics and fault handling. - Live demo of visualizer: inject a heavy job, show rebalancing and heat bloom. - Wrap with implications and future work (multi-star systems, economy models, emergent behaviour).

10. Next actions I can take right now (pick one)

1. Scaffold the repo (GitHub-ready) with initial files and CI.
 2. Flesh out the full physics engine (Python) with unit tests and example notebooks.
 3. Implement the scheduler core in Rust with a small job model and API.
 4. Build the Three.js visualizer skeleton with WebSocket client and shader examples.
 5. Draft the architecture whitepaper (2–3 pages) and demo script.
-

Credits & references

- Concepts: Dyson Sphere, Matrioshka Brain, Landauer limit, blackbody radiation.
- Tools & libs: Three.js, WebGPU, Rust, Go, Plotly, Prometheus/Grafana, PyTorch (if you add AI workloads).

If you want, tell me which of the "Next actions" (1–5) to start and I will generate the exact files and starter commits for that action.